



INSTITUTO POLITÉCNICO
NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Ingeniería en Inteligencia Artificial

Redes Neuronales y Aprendizaje Profundo

Práctica 1: Perceptron Multicapa y Clasificación

Profesor:

Saul de la O Torres

Alumno:

Velázquez Arrieta Eduardo Uriel

Fecha de entrega: 11 de Septiembre 2026

Índice

1. Introducción	2
2. Desarrollo Experimental	5
2.1. Implementación en TensorFlow	6
2.2. Implementación en PyTorch	9
2.3. Análisis Crítico de la Variación de Parámetros	13
3. Conclusiones	14
4. Referencias	15

Índice de figuras

1. Tabla de verdad de la compuerta AND.	2
2. Separabilidad lineal en la compuerta AND.	3
3. Tabla de verdad de la compuerta OR.	3
4. Separabilidad lineal en la compuerta OR.	3
5. Tabla de verdad de la compuerta XOR.	4
6. Incapacidad de separación lineal simple en la compuerta XOR.	4
7. Estructura del vector de características de entrada.	5
8. Distribución y formato tabular del conjunto de datos Iris.	6
9. Estructura orientada a objetos de la clase perceptrón en TensorFlow.	6
10. Métodos de inicialización y control de entrenamiento en TensorFlow.	7
11. Ejecución del entrenamiento con $\eta = 0,5$ y 500 épocas en TensorFlow.	7
12. Métricas de desempeño (Precision, Recall, F1-Score y Accuracy) para la Prueba 1.	8
13. Resultados con $\eta = 0,5$ y 50 épocas.	8
14. Resultados con $\eta = 0,7$ y 5 épocas.	9
15. Arquitectura y ciclo de entrenamiento orientado a objetos en PyTorch.	10
16. Resultados de ejecución en PyTorch (3000 épocas, $\eta = 0,1$).	11
17. Resultados de ejecución en PyTorch (500 épocas, $\eta = 0,5$).	12
18. Resultados de ejecución en PyTorch (50 épocas, $\eta = 0,7$).	13

1. Introducción

El campo de la Inteligencia Artificial (IA) abarca diversas disciplinas orientadas a modelar y resolver problemas complejos mediante la automatización del razonamiento y el aprendizaje computacional. Dentro del aprendizaje automático (*Machine Learning*), los problemas se estructuran principalmente en tres grandes paradigmas: clasificación (asignación de categorías discretas), regresión (predicción de valores continuos) y agrupamiento (*clustering*, segmentación por similitud). Si bien estos enfoques resultan sumamente eficientes en múltiples aplicaciones cotidianas, históricamente han mostrado limitaciones al intentar replicar los mecanismos subyacentes del procesamiento cognitivo y sensorial humano ante tareas aparentemente intuitivas.

Para comprender la evolución que condujo a los modelos modernos de aprendizaje profundo (*Deep Learning*), es fundamental estudiar los cimientos teóricos de las redes neuronales artificiales. El punto de partida histórico se remonta al modelo formal de la neurona artificial de McCulloch-Pitts y al perceptrón propuesto por Frank Rosenblatt, cuyo objetivo inicial consistía en modelar funciones lógicas básicas y resolver problemas de discriminación binaria mediante hiperplanos separadores.

A nivel conceptual, el análisis de compuertas lógicas como **AND**, **OR** y **XOR** ilustra la limitación fundamental de los perceptrones de una sola capa:

- **Compuerta AND y OR:** Representan problemas linealmente separables. Sus tablas de verdad pueden dividirse geoméricamente en el plano mediante una única línea recta (un hiperplano de dimensión 1), permitiendo que un perceptrón simple ajuste sus pesos sin dificultad.

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Figura 1: Tabla de verdad de la compuerta AND.

Al graficar las entradas de la compuerta AND, se observa que una función lineal del tipo $y = -x + \frac{3}{2}$ separa de manera exacta la clase 0 de la clase 1.

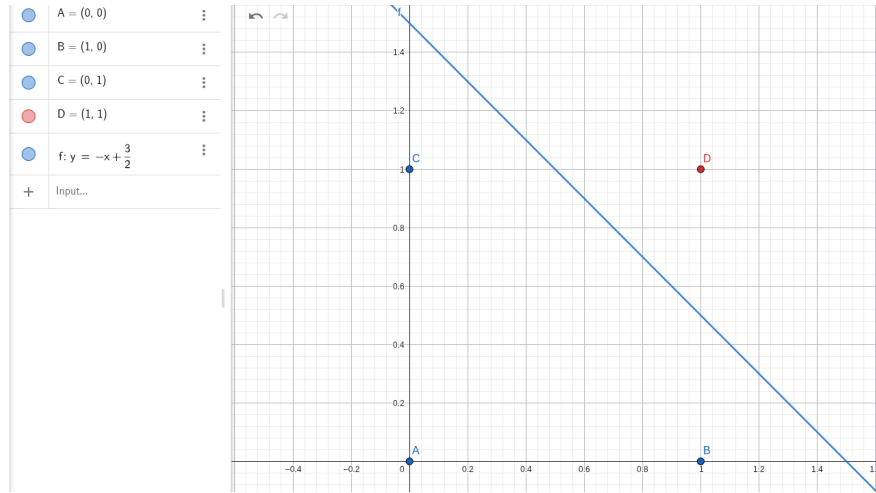


Figura 2: Separabilidad lineal en la compuerta AND.

- **Compuerta OR:** Al igual que la anterior, admite una solución lineal mediante el ajuste del sesgo o frontera con una función como $y = -x + \frac{1}{2}$.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Figura 3: Tabla de verdad de la compuerta OR.

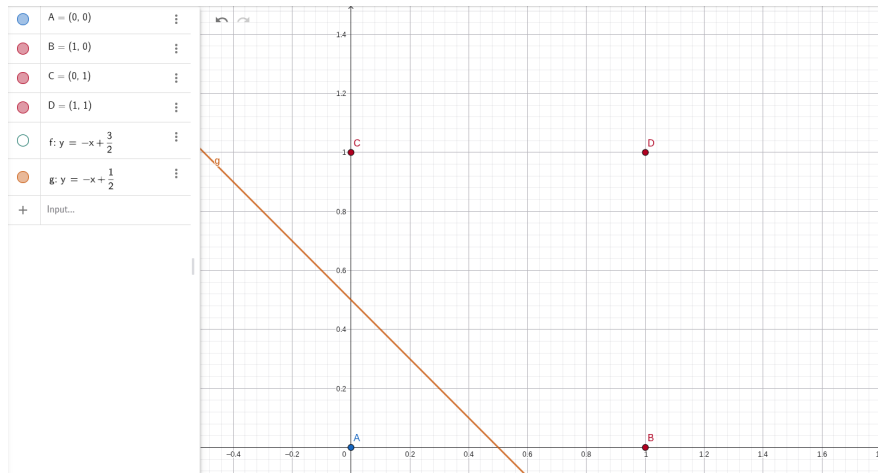


Figura 4: Separabilidad lineal en la compuerta OR.

- **Compuerta XOR:** Representa un problema no linealmente separable. Como se demostró en la literatura clásica de Minsky y Papert, un perceptrón simple es incapaz de resolver la función XOR debido a que los puntos de clases opuestas se entrelazan de tal forma que se requieren múltiples líneas de decisión (o capas ocultas) para lograr su clasificación.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Figura 5: Tabla de verdad de la compuerta XOR.

La imposibilidad de trazar un único hiperplano motivó la transición hacia arquitecturas capaces de aprender fronteras de decisión complejas, sentando las bases para el estudio de problemas multiclase del mundo real, tales como la clasificación del conjunto de datos **Iris** (compuesto por las especies *Setosa*, *Versicolor* y *Virginica*).

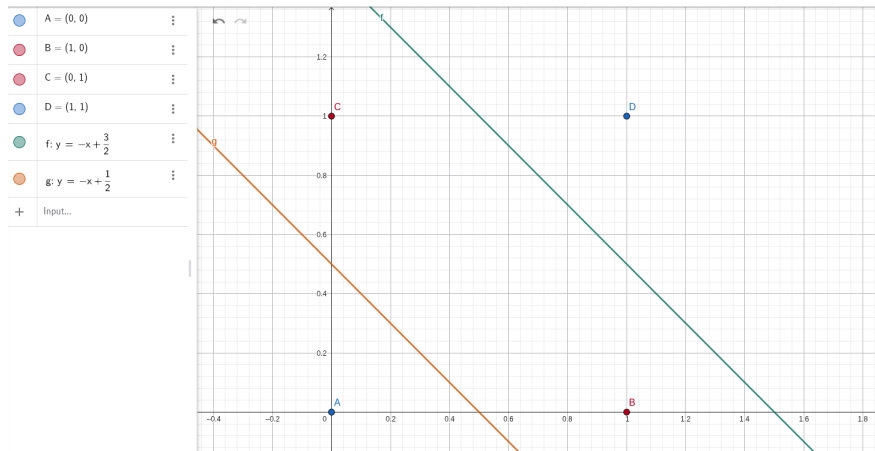


Figura 6: Incapacidad de separación lineal simple en la compuerta XOR.

2. Desarrollo Experimental

El propósito central de esta práctica es diseñar, implementar y evaluar arquitecturas de redes neuronales utilizando dos de los frameworks computacionales más potentes y adoptados en la industria y la investigación científica:

1. **TensorFlow**: Desarrollado por Google, destaca por su robustez en producción y su API de alto nivel orientada a la definición flexible de grafos y modelos orientados a objetos.
2. **PyTorch**: Desarrollado por Meta, sobresale por su paradigma de grafos dinámicos y su uso intensivo de tensores con diferenciación automática (*Autograd*), facilitando la depuración y el diseño modular.

Para evaluar el comportamiento de ambos entornos, se emplea el clásico conjunto de datos **Iris**. Este dataset se compone de vectores de cuatro características morfológicas estandarizadas:

A	B	C	D	X
---	---	---	---	---

Figura 7: Estructura del vector de características de entrada.

- A: Longitud del sépalo (*Sepal Length*)
- B: Ancho del sépalo (*Sepal Width*)
- C: Longitud del pétalo (*Petal Length*)
- D: Ancho del pétalo (*Petal Width*)
- X: Clase o etiqueta de la especie botánica (*Setosa*, *Versicolor*, *Virginica*)

El conjunto de datos consta de 150 instancias en total (50 por cada una de las tres clases). Adicionalmente, se integra un conjunto de validación independiente de 15 muestras nunca antes vistas por los modelos, permitiendo medir de forma rigurosa la capacidad de generalización fuera de muestra.

```

138 (6.4, 2.4, 5.2, 2.4) # 1. virginica
139 (6.4, 3.1, 5.5, 1.8) # 1. virginica
140 (6.4, 3.0, 4.8, 1.8) # 1. virginica
141 (6.4, 3.1, 5.4, 2.1) # 1. virginica
142 (6.7, 3.1, 5.6, 2.4) # 1. virginica
143 (6.9, 3.1, 5.2, 2.2) # 1. virginica
144 (5.8, 2.7, 5.1, 1.9) # 1. virginica
145 (6.8, 3.2, 5.9, 2.3) # 1. virginica
146 (6.7, 3.2, 5.7, 2.5)
147 (6.7, 3.0, 5.2, 2.3)
148 (6.3, 2.5, 5.0, 1.9)
149 (6.5, 3.0, 5.2, 2.0)
150 (6.2, 2.4, 5.4, 2.3)
151 (5.9, 3.0, 5.1, 1.8)
152
153
154 CLASS_IRIS = [
155     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
156     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
157     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
158     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
159     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
160     1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
161     1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
162     1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
163     1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
164     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0,
165     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0,
166     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0,
167     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0,
168     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0,
169     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0
170 ]
171
172 from collections import Counter
173
174 print(Counter(CLASS_IRIS))
175
176 Y_NUEVOS = [
177     (5.1, 3.4, 1.5, 0.2), (5.0, 3.5, 1.3, 0.3), (4.5, 2.3, 1.3, 0.3), (4.4, 3.2, 1.3, 0.2), (5.0, 3.5, 1.6, 0.6), # Setosa
178     (6.4, 2.9, 4.5, 1.1), (6.7, 2.8, 2.8, 1.0), (5.0, 2.4, 3.8, 1.1), (5.2, 2.4, 3.7, 1.0), (5.0, 2.7, 2.8, 1.2), # Versicolour
179     (6.9, 3.1, 5.4, 2.1), (6.7, 3.1, 5.6, 2.4), (6.9, 3.1, 5.1, 2.3), (5.8, 2.7, 5.1, 1.9), (6.8, 3.2, 5.9, 2.3) # Virginica
180 ]
181
182 Y_NUEVOS = [
183     0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
184     1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
185     2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0
186 ]
187
188
189

```

Figura 8: Distribución y formato tabular del conjunto de datos Iris.

2.1. Implementación en TensorFlow

La primera etapa experimental se desarrolló en **TensorFlow** empleando programación orientada a objetos. Esta estructura encapsula la lógica del modelo, facilitando la experimentación sistemática mediante la modificación de hiperparámetros.

- **Definición de la clase:** Se construye una clase orientada a objetos que hereda de las estructuras base del framework, configurando una capa densa de salida con activación *Softmax*, idónea para resolver problemas de clasificación multiclase (mapeando las salidas a probabilidades para las 3 clases).

```

1 from math import e
2 import numpy as np
3 import tensorflow as tf
4 from tensorflow.keras import Sequential
5 from tensorflow.keras.layers import Activation
6 import random
7 class Perceptron:
8     X = np.empty(0, dtype=np.float32)
9     # 2. Salidas esperadas para cada compuerta (Y)
10    Y_train = np.empty(0, dtype=np.float32)
11    learning_rate = 1.0
12    epochs = 100
13    verbose = 0
14    model = Sequential()
15    activation = Activation(None)
16
17    def __init__(self, *args):
18        if len(args) == 0:
19            self.learning_rate = 0.5
20            self.epochs = 400
21            self.verbose = 0
22            #dado que estamos haciendo clasificación multiclase tenemos que usar softmax
23            self.activation = Activation('softmax')
24        else:
25            print(args[0], " ", args[1], " ", args[2], " ", args[3])
26            self.learning_rate = float(args[0])
27            self.epochs = int(args[1])
28            self.verbose = int(args[2])
29            self.activation = Activation(args[3])
30            #print(self.learning_rate, " ", self.epochs, " ", self.verbose, " ", self.activation.get_config())
31

```

Figura 9: Estructura orientada a objetos de la clase perceptrón en TensorFlow.

- **Métodos principales:** La clase implementa funciones modulares para la carga de datos, definición de épocas, entrenamiento iterativo mediante optimizadores de descenso de gradiente y predicción final.

```

36 def definirSalida(self, entrenamiento):
37     self.Y_train = np.array(entrenamiento, dtype=np.float32)
38
39 def definirEpocas(self, epocas):
40     self.epochs = epocas
41
42 # 3. Función para crear y entrenar la célula
43 def entrenar_celula(self, nombre_compuerta):
44     self.compuerta = nombre_compuerta
45     print(f"\n--- Entrenando Compuerta {nombre_compuerta} ---")
46     # Estructura con 2 entradas (input_shape=[2])
47     # cambiamos a input_shape=[4] para el vector de 4 características que tenemos,
48     # units=3 dado que tenemos 3 clases, ocupamos 3 neuronas para clasificar de manera correcta
49     self.model = tf.keras.models.Sequential([
50         tf.keras.layers.Dense(units=3, input_shape=[4], activation=self.activation)
51     ])
52
53     # sparse_categorical_crossentropy para poder hacer clasificación multiclase
54     self.model.compile(
55         optimizer=tf.keras.optimizers.SGD(learning_rate=self.learning_rate),
56         loss='sparse_categorical_crossentropy',
57         metrics=['accuracy']
58     )
59     # Entrenar (epochs altos porque es un modelo muy simple con pocos datos)
60     print(self.learning_rate, " ", self.epochs, " ", self.verbose)
61     self.model.fit(self.X, self.Y_train, epochs=self.epochs, verbose=self.verbose)
62
63 def predecir(self, compuerta):
64     # Evaluar predicciones redondeadas
65     predicciones = self.model.predict(self.X)
66     # print(f"Predicciones para {self.compuerta}:")
67     # print(np.round(predicciones))
68     return np.round(predicciones)
69
70

```

Figura 10: Métodos de inicialización y control de entrenamiento en TensorFlow.

- **Prueba 1: Tasa de aprendizaje de 0.5 y 500 épocas.** Bajo esta configuración equilibrada, el modelo logra minimizar la función de pérdida de manera eficiente.

```

1 # 4. Ejecución del entrenamiento
2 # verbose [0, 1, 2]
3 perceptron = Perceptron(0.5, 500, 0, 'softmax')
4 # 1. Entradas comunes para dos variables (X1, X2)
5 X = IRIS
6 perceptron.definirVectorEntrada(X)
7 epocas = 500
8
9 # 2. Salidas esperadas para cada compuerta (Y)
10 y_and = CLASS_IRIS
11 y_or = CLASS_IRIS
12 y_xor = CLASS_IRIS
13
14
15 perceptron.definirSalida(y_and)
16 perceptron.definirEpocas(epocas)
17 perceptron.entrenar_celula("AND")
18 and_res = perceptron.predecir("AND")
19
20
21 perceptron.definirSalida(y_or)
22 perceptron.definirEpocas(epocas)
23 perceptron.entrenar_celula("OR")
24 or_res = perceptron.predecir("OR")
25
26
27 perceptron.definirSalida(y_xor)
28 perceptron.definirEpocas(epocas)
29 perceptron.entrenar_celula("XOR")
30 xor_res = perceptron.predecir("XOR")
31
32
33 # Ejecución
34 and_res1 = crear_matriz(and_res)
35 or_res1 = crear_matriz(or_res)
36 xor_res1 = crear_matriz(xor_res)
37
38 and_verdaderos = evaluar(and_res1)
39 or_verdaderos = evaluar(or_res1)
40 xor_verdaderos = evaluar(xor_res1)
41
42 print("\n\n Compuerta AND ,.5 tasa de aprendizaje, 500 epocas \n\n")
43 matrix_confusion(and_verdaderos,3)
44 print("\n\n Compuerta OR ,.5 tasa de aprendizaje, 500 epocas \n\n")
45 matrix_confusion(or_verdaderos,3)

```

Figura 11: Ejecución del entrenamiento con $\eta = 0,5$ y 500 épocas en TensorFlow.

Las métricas obtenidas reflejan un rendimiento sobresaliente, clasificando correctamente el 100 % de las muestras de validación externas.

```

--- Evaluando 15 DATOS NUEVOS con modelo TENSORFLOW ---
Muestra 01 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 04 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 05 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 06 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 12 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 13 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 14 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 15 | Esperado: 2 | Predicción: 2 -> ✓ Correcto

-> Aciertos en datos nunca antes vistos: 15/15 (100.00%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 1.0

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	1.00	1.00	1.00	1.00	1.00
Clase 2:	1.00	1.00	1.00	1.00	1.00
AVG:	1.00	1.00	1.00	1.00	1.00

Figura 12: Métricas de desempeño (Precision, Recall, F1-Score y Accuracy) para la Prueba 1.

Análisis de resultados: El éxito de esta prueba radica en la sinergia entre un número suficiente de épocas y una tasa de aprendizaje moderada, permitiendo que el optimizador encuentre los mínimos globales sin sobrepasar la región de convergencia.

- **Prueba 2: Tasa de aprendizaje de 0.5 y 50 épocas.** Reduciendo el número de iteraciones a una décima parte.

```

--- Evaluando 15 DATOS NUEVOS con modelo TENSORFLOW ---
Muestra 01 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 04 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 05 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 06 | Esperado: 1 | Predicción: 2 -> ✗ Incorrecto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 12 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 13 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 14 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 15 | Esperado: 2 | Predicción: 2 -> ✓ Correcto

-> Aciertos en datos nunca antes vistos: 14/15 (93.33%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 0.9333333333333333

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	1.00	0.80	0.89	1.00	0.93
Clase 2:	0.83	1.00	0.91	0.90	0.93
AVG:	0.94	0.93	0.93	0.97	0.96

Figura 13: Resultados con $\eta = 0,5$ y 50 épocas.

Análisis: Se observa una degradación notable en las métricas. La falta de épocas impide que los pesos sinápticos converjan a los valores óptimos, evidenciando un problema de subajuste (*underfitting*).

- **Prueba 3: Tasa de aprendizaje de 0.7 y 5 épocas.** Configuración altamente restrictiva en tiempo y agresiva en actualización.

```

--- Evaluando 15 DATOS NUEVOS con modelo TENSORFLOW ---
Muestra 01 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 1 -> ✗ Incorrecto
Muestra 04 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 05 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 06 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 12 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 13 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 14 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 15 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto

-> Aciertos en datos nunca antes vistos: 9/15 (60.00%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 0.6

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	0.80	0.89	1.00	0.93
Clase 1:	0.45	1.00	0.62	0.40	0.60
Clase 2:	0.00	0.00	0.00	1.00	0.67
AVG:	0.48	0.60	0.50	0.80	0.73

Figura 14: Resultados con $\eta = 0,7$ y 5 épocas.

Análisis: El rendimiento colapsa. Combinar un número ínfimo de épocas con una tasa de aprendizaje grande genera oscilaciones violentas en el espacio de pesos, impidiendo cualquier forma de aprendizaje útil.

2.2. Implementación en PyTorch

La segunda fase implementa la misma arquitectura bajo **PyTorch**, utilizando módulos basados en ‘nn.Module’ y optimizadores estocásticos con tensores explícitos.

- **Definición de la clase RedNeuronal:**

```

class RedNeuronal:
    # =====
    # 2. CONFIGURACION DE DATOS
    # =====
    X = torch.tensor(IRIS, dtype=torch.float32)
    Y = torch.tensor(CLASS_IRIS, dtype=torch.long)

    # =====
    # 3. ENTRENAMIENTO Y EVALUACION
    # =====
    # Se usan exactamente los mismos parametros para las tres compuertas

    def ejecutarRed(self, epochs, learning_rate):

        print(f"El entranamiento se hizo con {epochs} epocas con una tasa de aprendizaje de {learning_rate}")
        model = Perceptron() # instancia a la clase
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.SGD(model.parameters(), lr=learning_rate)

        # Ciclo de entrenamiento
        for epoch in range(epochs): # para cada epoca
            outputs = model(self.X) # tomamos el vector
            loss = criterion(outputs, self.Y) # tomamos los criterios para la perdida

            optimizer.zero_grad() # optimizador del gradiente cero
            loss.backward() # calcula la perdida
            optimizer.step() # optimiza por paso

        # Evaluacion y diagnostico del aprendizaje si hay fallo
        print(f"Resultado Final:")
        with torch.no_grad():
            predicciones = model(self.X) # vuelve a tomar el vector
            predicciones = torch.argmax(predicciones, dim=1) # hace la prediccion
            # Diagnostic prints to understand the ValueError
            print(f"DEBUG: Type of 'predicciones' before return: {type(predicciones)}")
            print(f"DEBUG: Type of 'model' before return: {type(model)}")
        return predicciones, model # Return both predictions and the trained model

```

Figura 15: Arquitectura y ciclo de entrenamiento orientado a objetos en PyTorch.

- **Prueba con 3000 épocas y tasa de 0.1:** Al extender las épocas y refinar el paso de aprendizaje, PyTorch logra una precisión global del 98 % en entrenamiento y un 100 % impecable en los 15 datos nuevos de validación.

```

El entrenaiento se hizo con 3000 epocas con una tasa de aprendizaje de 0.1
Resultado Final:
DEBUG: Type of 'predicciones' before return: <class 'torch.Tensor'>
DEBUG: Type of 'model' before return: <class '__main__.Perceptron'>
Accuracy global: 0.98

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	0.98	0.96	0.97	0.99	0.98
Clase 2:	0.96	0.98	0.97	0.98	0.98
AVG:	0.98	0.98	0.98	0.99	0.99

```

--- Evaluando 15 DATOS NUEVOS con modelo PYTORCH ---
Muestra 01 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 04 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 05 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 06 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 12 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 13 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 14 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 15 | Esperado: 2 | Predicción: 2 -> ✓ Correcto

-> Aciertos en datos nunca antes vistos: 15/15 (100.00%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 1.0

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	1.00	1.00	1.00	1.00	1.00
Clase 2:	1.00	1.00	1.00	1.00	1.00
AVG:	1.00	1.00	1.00	1.00	1.00

Figura 16: Resultados de ejecución en PyTorch (3000 épocas, $\eta = 0,1$).

- Prueba con 500 épocas y tasa de 0.5:

```

El entranamiento se hizo con 500 epocas con una tasa de aprendizaje de 0.5
Resultado Final:
DEBUG: Type of 'predicciones' before return: <class 'torch.Tensor'>
DEBUG: Type of 'model' before return: <class '__main__.Perceptron'>
Accuracy global: 0.98

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	0.98	0.96	0.97	0.99	0.98
Clase 2:	0.96	0.98	0.97	0.98	0.98
AVG:	0.98	0.98	0.98	0.99	0.99

```

--- Evaluando 15 DATOS NUEVOS con modelo PYTORCH ---
Muestra 01 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 04 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 05 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 06 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 12 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 13 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 14 | Esperado: 2 | Predicción: 2 -> ✓ Correcto
Muestra 15 | Esperado: 2 | Predicción: 2 -> ✓ Correcto

-> Aciertos en datos nunca antes vistos: 15/15 (100.00%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 1.0

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	1.00	1.00	1.00	1.00
Clase 1:	1.00	1.00	1.00	1.00	1.00
Clase 2:	1.00	1.00	1.00	1.00	1.00
AVG:	1.00	1.00	1.00	1.00	1.00

Figura 17: Resultados de ejecución en PyTorch (500 épocas, $\eta = 0,5$).

■ Prueba con 50 épocas y tasa de 0.7:

```

El entrenamiento se hizo con 50 épocas con una tasa de aprendizaje de 0.5
Resultado Final:
DEBUG: Type of 'predicciones' before return: <class 'torch.Tensor'>
DEBUG: Type of 'model' before return: <class '__main__.Perceptron'>
Accuracy global: 0.4066666666666667

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	0.22	0.36	1.00	0.74
Clase 1:	0.36	1.00	0.53	0.11	0.41
Clase 2:	0.00	0.00	0.00	1.00	0.67
AVG:	0.45	0.41	0.30	0.70	0.60

```

--- Evaluando 15 DATOS NUEVOS con modelo PYTORCH ---
Muestra 01 | Esperado: 0 | Predicción: 1 -> ✗ Incorrecto
Muestra 02 | Esperado: 0 | Predicción: 0 -> ✓ Correcto
Muestra 03 | Esperado: 0 | Predicción: 1 -> ✗ Incorrecto
Muestra 04 | Esperado: 0 | Predicción: 1 -> ✗ Incorrecto
Muestra 05 | Esperado: 0 | Predicción: 1 -> ✗ Incorrecto
Muestra 06 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 07 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 08 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 09 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 10 | Esperado: 1 | Predicción: 1 -> ✓ Correcto
Muestra 11 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 12 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 13 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 14 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto
Muestra 15 | Esperado: 2 | Predicción: 1 -> ✗ Incorrecto

-> Aciertos en datos nunca antes vistos: 6/15 (40.00%)

Métricas Globales (Solo para los 15 datos nuevos):
Accuracy global: 0.4

```

	Precision	Recall	F1-Score	Specificity	Accuracy
Clase 0:	1.00	0.20	0.33	1.00	0.73
Clase 1:	0.36	1.00	0.53	0.10	0.40
Clase 2:	0.00	0.00	0.00	1.00	0.67
AVG:	0.45	0.40	0.29	0.70	0.60

Figura 18: Resultados de ejecución en PyTorch (50 épocas, $\eta = 0,7$).

2.3. Análisis Crítico de la Variación de Parámetros

El comportamiento observado en ambos frameworks demuestra que el rendimiento de un modelo neuronal no es estático, sino altamente sensible a dos hiperparámetros críticos:

1. **Tasa de Aprendizaje (η):** Regula la magnitud de las correcciones aplicadas a los pesos sinápticos en cada iteración del gradiente.
 - Si es *demasiado baja*, el entrenamiento requiere un número impráctico de iteraciones y puede estancarse en mínimos locales secundarios.
 - Si es *demasiado alta* (como 0,7 en pocas épocas), el optimizador sobrepasa constantemente el punto óptimo de convergencia, provocando divergencia numérica y oscilaciones erráticas en la función de pérdida.
2. **Número de Épocas:** Define el horizonte temporal de aprendizaje del modelo sobre el conjunto de datos completo. Un déficit de épocas trunca el proceso de minimización del error, mientras que un balance adecuado permite estabilizar los pesos sin incurrir necesariamente en sobreajuste debido a la simplicidad del dataset analizado.

3. Conclusiones

A lo largo del desarrollo de esta práctica se cumplieron satisfactoriamente los objetivos planteados, logrando comprender y contrastar el comportamiento operativo de los perceptrones simples y multicapa aplicados a problemas de clasificación multiclase. Mediante el uso de TensorFlow y PyTorch, se evidenció que la implementación bajo un paradigma orientado a objetos otorga una alta modularidad, escalabilidad y claridad en la gestión de flujos de datos.

Asimismo, los experimentos demostraron que el éxito en el rendimiento predictivo (reflejado en métricas de precisión, *recall* y *F1-score*) depende de forma directa de la sintonización adecuada de los hiperparámetros de entrenamiento. Se comprobó empíricamente que configuraciones desbalanceadas —como tasas de aprendizaje agresivas combinadas con un número insuficiente de épocas— destruyen la capacidad de convergencia del modelo, arrojando resultados deficientes, en contraste con configuraciones estables que alcanzan una generalización óptima del 100 % en datos no vistos.

4. Referencias

- “Iris Species”, Kaggle Dataset Repository. Accedido el 7 de septiembre de 2026. [En línea]. Disponible: <https://www.kaggle.com/datasets/uciml/iris>
- A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library”, en *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, pp. 8026-8037.
- M. Abadi et al., “TensorFlow: A system for large-scale machine learning”, en *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 265-283.
- F. Chollet, *Deep Learning with Python*, 2.a ed. Manning Publications, 2021.
- S. Haykin, *Neural Networks and Learning Machines*, 3.a ed. Pearson, 2008.
- F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain”, *Psychological Review*, vol. 65, no. 6, pp. 386-408, 1958.